

1.

(1) 设计差异:

O(1) 是“分配时间片”，时间片耗尽就去后台等待；CFS 是“模拟理想多任务处理器”，谁在公平线上落后了，谁就优先运行，从而动态地消除了对“交互式任务”的手动猜想。

CFS 放弃了固定时间片的概念，引入了虚拟运行时间。它通过一棵红黑树维护就绪进程，红黑树的键值即为虚拟运行时间。调度器总是选择树中最左侧（虚拟运行时间最小）的任务运行。任务运行后，其虚拟运行时间增加，增加速度与权重（由 Priority 映射）成反比。

CFS 无需区分活跃与过期，实现了动态平衡，并且提供了连续的、排序好的任务视图。

(2) 是后者。

在无限小的时间轴上，每个进程按权重比例获取 CPU 时间。

最小粒度指的是每个任务被分配的最小时间片，是防止因任务过多导致每个任务跑的时间太短而被频繁切出的设置。（考虑到上下文切换有代价（Cache 失效、寄存器保存），时间片不能被无限切分。）

而目标延迟要求所有任务在一个周期内至少运行一次，防止某个任务被忽视或者是拖延。

这是一种针对现代交互式负载优化的工程近似公平而非绝对数学公平。

(3) 不。

在极高负载的批处理系统中 O(1) 效果更优。此时 CFS 插入/删除任务的时间复杂度是 $O(\log N)$ ，而 O(1) 时间复杂度始终是 $O(1)$ 。当 N 极大时，红黑树的维护开销（树旋转、缓存不命中）会显著增加调度本身的 CPU 占用。

(4) 没有真正解决

它将原先 O(1) 的复杂的奖励公式简化为 latency, granularity 等参数的优化，消除了 O(1) 中复杂的启发式优先级升降算法（那个算法导致了不可预测的交互行为），从架构上更简洁。

但它并未完全通过算法消除启发式，而是将参数收敛到了几个关键阈值上。

挑战：

在 Big.LITTLE（异构）架构下，CFS 的公平定义失效。

CFS 的基本假设是：“1ms 的 CPU 时间对所有核心都是等价的”。但在异构架构中，这一假设并不成立，“1ms 的大核时间”可能不等于“1ms 的小核时间”，具体有如下新挑战：

- (1) 算力不均：不同类型的核运行相同时间完成的工作量不同
 - (2) 公平性问题：按照时间公平分配任务给大小核将会造成不同程度的时间不协调。
- (应对措施是 EAS：引入了与算力相关的参数，从分配 CPU 时间转向了分配处理能力)

2.

(1) 将各阶段及其根本性矛盾画表如下：

演进阶段	核心机制	驱动的根本性矛盾
早期全局就绪队列	多个 CPU 共享一个任务链表	锁竞争矛盾： 所有 CPU 为了取任务都要锁住同一个队列，导致核数增加时性能反而下降。

演进阶段	核心机制	驱动的根本性矛盾
O(1) 的 Per-CPU 队列	每个 CPU 拥有独立的运行队列	均衡与局部性的矛盾 ：虽然解决了锁竞争，但会导致某些核空闲、某些核过载
CFS 与调度域	引入分层拓扑架构	拓扑感知矛盾 ：简单的任务平均分配忽略了“核心距离”。跨核迁移会杀掉 Cache，跨 Socket 迁移会触发 NUMA 惩罚。
EAS 能效感知调度	引入能量模型	性能与续航的矛盾 ：在异构架构下，低负载任务放在大核是浪费，高负载任务放小核是低效。

(2)

运作流程：

(1) CPU 定时检查所属调度域的均衡状态，当一个 CPU 的就绪队列变空时，它会主动去别的核拉取任务。

`find_busiest_queue` 的搜索过程：

- (a) 调度器遍历当前 CPU 所在的调度域
- (b) 计算每个调度组的平均负载
- (c) 锁定负载最高且超过平均值的队列，并计算需要迁移的“任务负载量”

代价

- (1) Cache 冷启动：每个 CPU 有自己的 L1/L2 Cache。进程迁移后，原有缓存失效，新核心必须从主存或 L3 重新加载数据，导致短期内“每指令周期”骤降。
- (2) TLB 刷新：跨核往往伴随着页表缓冲的重新填充。
- (3) NUMA 远程访存：在多路服务器上，跨 Socket 迁移意味着进程访问原有的内存区域变成了“远程访问”，延迟增加数倍。

权衡：

- (1) 亲和性惩罚：除非目标核心与源核心的负载差异超过一定阈值（不均衡程度超过限度），否则不执行迁移。
- (2) 分层迁移开销：在同一核心的两个 HT（超线程）间移动任务代价极小；但在两个 NUMA 节点间移动任务，代价极大。
- (3) 不可热点进程：正在频繁运行、Cache 热度极高的进程，调度器会标记为尽量不迁移。

3.

(1) 可剥夺的优先级调度算法：

P1→P2→P1→P3→P4

平均周转时间：13ms，平均带权周转时间：2.2

(2) 基于优先级的时间片轮转算法

P1(0-3)→P2(3-5)→P1(5-6)→P4(6-7)→P3(7-9)→P4(9-12)→P1(12-13)→P3(13-14)→P4(14-16)→P2(16-18)→P4(18-20)→P2(20-22)→P4(22-26)

平均周转时间：15.25ms，平均带权周转时间：2.377

4.

(1) 是

解决方案：

Linux 引入了 NAPI 机制，从中断驱动转为轮询驱动：

1. 当第一个数据包到达时，网卡触发中断。
2. 内核在处理该中断时，直接通知网卡：“暂时不要再发中断”。
3. 内核调度一个专门的内核线程或利用软中断上下文，以轮询方式从网卡的 DMA 缓存中批量读取数据包。
4. 内核限制每次轮询处理的数据包数量。当处理完这些包后，内核会主动让出 CPU 资源给用户程序。
5. 只有当缓存区的数据包全部处理完毕，内核才会重新开启网卡的硬中断。

(2)

优：当系统的核心矛盾在于“计算任务对时间抖动极度敏感”时，这是最优选。原因如下：

(1) 消除非预期抢占：对于实时系统或高频交易系统，进程需要在一个极短的时间窗口内完成计算。如果此时 CPU 收到网卡或磁盘中断，流水线会强制清空，跳转到内核态。中断带来的这种“随机停顿”是无法预估的。将中断隔离在 CPU0，可以保证其他核心的计算进程拥有纯净的执行时间流。

(2) 最大化缓存命中率：中断处理程序会频繁访问内核代码段地址和 I/O 缓冲区，这会冲掉核心 L1/L2 Cache 中原本属于用户进程的热点数据。如果将中断固定在 CPU0，其他核心的 Cache 污染将降至最低，非常适合高性能计算或大规模矩阵运算。

(3) 简化锁竞争逻辑：在某些老旧内核或驱动设计中，中断处理可能涉及全局锁。集中在同一 CPU 处理可以减少跨核同步导致的 CPU 周期空转。

劣：当系统的核心矛盾在于“I/O 总吞吐量”时，这是最劣选。

(1) 单点瓶颈：随着万兆网卡的普及，单位时间内产生的中断包量极大。如果只有一个核心处理中断，CPU0 会极速达到 100% 负载（主要是软中断 `si` 占用）。此时即使 CPU1~N 非常空闲，系统也会因为延迟响应 I/O 而导致网络丢包或磁盘响应超时。

(2) 跨核通信成本：

- IPI（处理器间中断）：若 CPU1 上的进程发起 `read` 请求，中断却发生在 CPU0，CPU0 处理完后需通过 IPI 唤醒 CPU1，这引入了额外的指令开销。
- Cache Bouncing（缓存颠簸）：数据由 CPU0 接收并写入内存，但由 CPU1 处理。这种跨核心的数据搬移会导致 CPU 缓存一致性协议频繁工作，大幅拉高内存访问时延。
- NUMA 拓扑冲突：在多路物理 CPU 服务器上，如果网卡插在 CPU1 对应的 PCIe 插槽，却强行将中断绑定在 CPU0，数据将不得不跨越 QPI/UPI 总线，造成严重的 NUMA 远程访存延迟。

权衡：考虑建立一个简单的数理决策模型：

设系统总负载为 L ，计算任务所需 CPU 时间为 C ，中断处理所需时间为 I 。

- 如果 $I \ll 1$ 个核心的处理能力，且对单位任务时延波动要求 $< 1\%$ ：绑定 CPU0 为优。通过牺牲 CPU0 的利用率来置换其他核心的绝对纯净。
- 如果 $I > 0.5$ 且系统任务是高并发请求：绑定 CPU0 为劣。此时应开启 RSS（接收端调节）和 RPS（软中断分布式处理），利用 `irqbalance` 将中断均匀散布在所有核心上。